LOOPGYM: TARGET-HIDDEN LOOP VERIFICATION WITH A SHARED ROLLOUT–HOUDINI FRAMEWORK

Anonymous authors
Paper under double-blind review

ABSTRACT

Verifying a loop requires more than producing a predicate that is inductive: the predicate must retain enough information to establish the condition required after the loop. Most invariant-generation systems expose this postcondition or assertion to the generator, allowing candidates to be tailored to, or copied from, the target. We study *target-hidden loop verification*. The generator sees the precondition, loop guard, and body but not the target Q; it emits ACSL loop invariants I, which are accepted only when Frama-C/WP proves both their inductiveness and the original exit obligation $I \wedge \neg B \Rightarrow Q$. LOOPGYM organizes learning and inference around one rollout–Houdini framework. Both form groups of invariant candidates, merge or repair their clauses, and apply the same Frama-C/WP Houdini operator. The training objective attaches dense credit to this core: a rollout is scored by standalone rejection, ordered within-rollout redundancy control, and leave-one-rollout-out group contribution on audited target-free traces. A repair is scored by its gain over the merged pool. Every policy response obeys a configurable clause cap, with explicit overflow cost. Inference executes the same rollout, pool-growth, feedback, and pruning stages, then restores Q for the final proof. This design aligns the model actions and prover-backed survivor semantics optimized during training with those executed at deployment.

1 INTRODUCTION

Loop invariants are the linchpin of deductive program verification. Ever since the inductive-assertion method of Floyd (Floyd, 1967) and the axiomatic semantics of Hoare (Hoare, 1969), verifying a loop `while` $(B)\, S$ against a contract $\{P\} \cdot \{Q\}$ has reduced to exhibiting an *inductive invariant* I satisfying initiation ($P \Rightarrow I$), consecution ($\{I \wedge B\}\, S\, \{I\}$), and safety ($I \wedge \neg B \Rightarrow Q$). The goal is not merely to find a predicate that satisfies the first two obligations. An invariant such as true may be inductive but carries no information to the loop exit. Useful loop reasoning must preserve enough information to establish the required post-loop condition Q. This final obligation is the distinction between generating arbitrary invariants and completing loop verification.

Existing methods use abstract domains, symbolic constraints, counterexamples, or learned generators to search for such invariants (Cousot & Cousot, 1977; Colón et al., 2003; Garg et al., 2016; Si et al., 2020; Kamath et al., 2023; Cao et al., 2025). Many learning-based systems expose Q to the generator and use its proof as feedback. This is effective for a fixed verification instance, but it creates a shortcut: a model can tailor candidates to the visible assertion without learning the relations and bounds that explain the loop. Postcondition-derived invariant mutation makes this dependence explicit (Furia & Meyer, 2010).

Motivating example. Consider `NLA_lipus/10.c`, abridged below:

```
int x = 1, y = 1, c = 1;
while (c < k) {
  c = c + 1;
  x = x*z + 1;
  y = y*z;
}
/*@ assert 1+x*z-x-z*y == 0; */
```

The assertion itself is established by the initial state and preserved by the loop body. When it is visible, invariant generation collapses to copying the target: `loop invariant 1+x*z-x-z*y == 0;`. The resulting clause is inductive and proves the assertion without revealing why this relation follows from the two coupled recurrences. When the assertion is hidden, the same clause must instead be recovered from the assignments to `x` and `y`. This small example captures the distinction we study: target-visible verification can make invariant inference trivial even when the underlying relation is nonlinear.

Target-hidden loop verification. We study a stricter setting. The full verification instance contains a precondition, loop, and post-loop target Q, but the policy receives only the precondition, guard, body, and in-scope variables. Assertions and `ensures` clauses are removed before generation, refinement, and Houdini pruning. The policy emits ACSL loop invariants, not a new postcondition. Only at the final gate does LOOPGYM restore the original target and ask Frama-C/WP to prove all invariant obligations and the post-loop condition. For a standard `while` loop, acceptance therefore requires

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\}, \qquad I \wedge \neg B \Rightarrow Q.$$

The hidden target determines success, while the generator cannot read or paraphrase it. The task is consequently not "generate any loop invariant," but "recover loop conditions that carry enough information to the exit to prove a held-out requirement."

One framework for training and inference. The central design of LOOPGYM is a shared rollout–Houdini loop, rather than separate training and deployment procedures. Given the masked program, both paths sample a group of invariant rollouts, normalize and merge their clauses, optionally grow the pool through stateless WP-guided repairs, and invoke the same Houdini-style Frama-C/WP operator (Flanagan & Leino, 2001; Kirchner et al., 2015) to retain an inductive subset. Only the consumer of those survivors changes. During training, Houdini survivors are scored for behavioral coverage: generation rollouts receive standalone and group-marginal credit, and repairs receive credit for improving the merged pool. Within each generation rollout, clauses are traversed in model-output order: an admitted clause is penalized only when it rejects no negative trace beyond those already rejected by earlier clauses. During inference, the final Houdini survivors are inserted into the original program and checked against the restored Q. Thus reward is coupled to exactly the rollout, pool-growth, feedback, and pruning semantics used at deployment; it is not an auxiliary objective attached to a different inference pipeline.

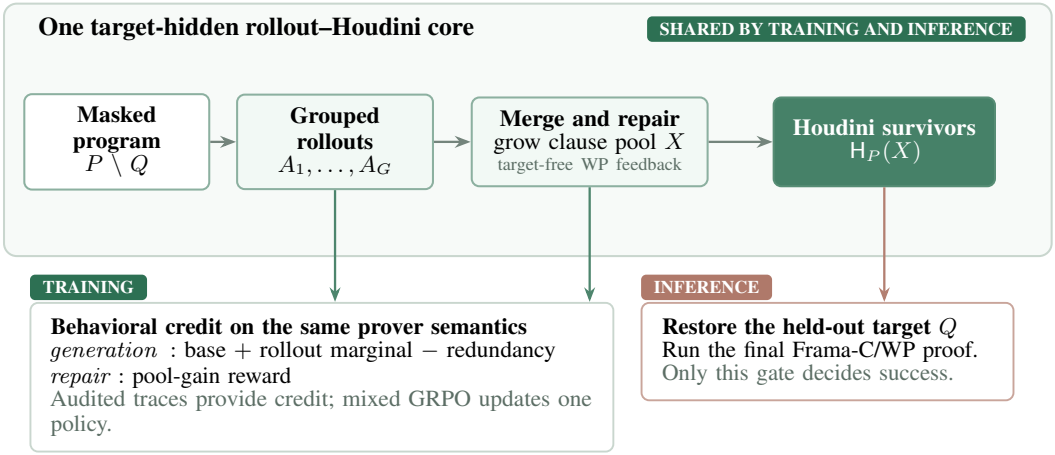

Figure 1: **Training and inference execute the same target-hidden rollout–Houdini core.** Both form rollout groups, grow a merged pool with target-free feedback, and retain clauses through the same Frama-C/WP Houdini operator. Training turns these operations into generation and refinement rewards; inference instead restores Q and applies the final proof gate. The execution sampler supplies training credit but is never called at inference.

Hiding Q removes the binary target-verification reward, while inductiveness alone remains too weak because tautologies and loose bounds pass. The execution sampler therefore constructs relational

perturbations, continuations beyond a genuine exit, and sampled-range escapes without consulting Q. Candidates contradicted by observed reachability, feasible fresh entries, or incomplete state coverage are suppressed. These audited traces provide the dense strength signal attached to the shared framework, while the held-out target remains the authoritative final criterion.

Contributions. This paper makes the following contributions:

- **A target-hidden verification task.** We separate the policy view from the verification instance: the model generates invariants without Q, while acceptance still requires proving Q after the loop (§3).
- **A shared rollout–Houdini framework.** Training and inference use the same grouped rollouts, growing clause pools, stateless WP feedback, and prover-backed Houdini survivors; training attaches rewards, while inference attaches the restored final proof obligation.
- **An execution-grounded objective.** Generation receives dominant standalone coverage, ordered clause-level redundancy control, and positive cross-rollout marginal credit; refinement receives its gain over the shared pool. Response caps and explicit redundancy/overflow costs prevent clause accumulation, while three conservative negative families provide target-free strength evidence (§4).
- **An auditable evaluation contract.** We define controlled target-visibility, equal-budget inference, and reward ablations, and validate the implemented signal with known-quality discrimination and a full-suite label audit (§5).

2 Related Work

Loop-invariant generation. Classical techniques infer invariants with abstract domains, templates, interpolation, or recurrences (Cousot & Cousot, 1977; Colón et al., 2003; McMillan, 2003; Kincaid et al., 2017); data-driven systems instead infer likely relations from traces and then filter or prove them (Ernst et al., 2007; Nguyen et al., 2012; Garg et al., 2016). Recent LLM systems broaden the candidate language. LOOPY generates ACSL clauses, filters them with Houdini, and repairs failures using Frama-C feedback (Kamath et al., 2023); CLAUSE2INV generates clauses and combines them under counterexample guidance (Cao et al., 2025). IRANK addresses the cost of sampling by contrastively re-ranking LLM-generated invariants before verifier calls (Chakraborty et al., 2023). These systems improve candidate generation, combination, or ordering for a supplied verification problem. By contrast, LOOPGYM withholds the post-loop target Q throughout generation and training, then accepts an invariant set only if it is inductive *and* proves the restored obligation $I \wedge \neg B \Rightarrow Q$. Houdini (Flanagan & Leino, 2001) is therefore a sound pruning component in our pipeline, not the generation objective: it can remove non-inductive clauses from a fixed pool, but cannot determine whether the surviving pool retains enough information for an unseen Q. Our additional distinction is alignment: grouped rollout construction and Houdini survival define one shared core for both policy training and deployment inference.

Specification generation. LLM-based tools also synthesize artifacts broader than loop invariants. AUTOSPEC decomposes C programs, repeatedly samples ACSL contracts and annotations, and retains verifier-accepted clauses (Wen et al., 2024); SPECGEN combines LLM proposals with mutation in Java/JML (Ma et al., 2025). SESPEC uses symbolic execution to expose path-sensitive strongest-postcondition information and generate goal-independent, verifier-checked specifications for C (Yang et al., 2026). These systems aim to recover function-level behavioral specifications, possibly including preconditions, postconditions, predicates, and loop annotations. LOOPGYM studies a narrower but deliberately controlled problem: the output is an ACSL loop-invariant set, the downstream target is hidden rather than generated, and success is measured by whether the set is sufficient for that held-out target. Thus our focus is not general contract recovery, but learning target-agnostic loop facts that remain useful when a concrete verification obligation is later restored.

Reinforcement learning from formal feedback. Earlier invariant learners use solver feedback to train policies or differentiable predicates (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020). RE:FORM reduces human priors in Dafny generation through scalable data curation and verifier-guided RL (Yan et al., 2025). SPECRL goes beyond binary verifier acceptance: it generates

impossible input–output pairs and rewards specifications that reject them, providing a completeness signal for Dafny specification synthesis (Huang et al., 2026). LOOPGYM shares the idea that formal validity alone cannot distinguish a useful specification from a weak one, but specializes it to hidden-target loops. Its negatives are relational perturbations, over-run exit continuations, and sampled-range escapes over internal loop states; conservative feasibility and coverage guards suppress uncertain labels, a broader execution audit checks for collisions, and Frama-C/WP separately proves inductiveness. Generation reward separates ordered clause redundancy within one rollout from leave-one-rollout-out complementarity across rollouts; repair reward measures the gain of a capped response over the same merged pool consumed by inference. The held-out Q is never used as a feature or reward label and appears only in final acceptance.

3 PROBLEM SETTING

3.1 TARGET-HIDDEN EXIT SUFFICIENCY

Fix a verification instance (P, Q), where $P = (\phi_{\text{pre}}, B, S)$ contains the precondition, loop guard, and body, and Q is a post-loop assertion or function postcondition. Let $\mathcal{R}(P)$ be the states reachable at the loop head from an initial state satisfying ϕ_{pre}. A predicate I is *inductive* when

$$\phi_{\text{pre}} \Rightarrow I \qquad \text{and} \qquad \{I \wedge B\}\, S\, \{I\}.$$

Inductiveness gives a sound over-approximation of $\mathcal{R}(P)$, but does not make it useful for a particular verification task. We call I *exit-sufficient* for Q when

$$I \wedge \neg B \Rightarrow Q.$$

Definition 1 (Target-hidden loop verification)**.** *The verification framework receives (P, Q), but the policy receives only P. It must generate an invariant set I without observing Q, and succeeds only when I is both inductive and exit-sufficient for the original Q.*

The restriction is on information flow, not output syntax: both visible and hidden systems emit ACSL `loop invariant` clauses. In the visible setting, however, Q may itself be inductive and can simply be copied into the loop, as in `NLA_lipus/10.c`. Target hiding removes this shortcut. It also makes binary verification unusable as a training reward, since querying the held-out target during RL would leak the answer. LOOPGYM instead rewards target-free behavioral strength and retains the original Q as the sole final acceptance criterion.

3.2 A SHARED CANDIDATE-PROCESSING OPERATOR

For a rollout group $\mathcal{A} = \{A_1, \ldots, A_G\}$ and response-cap hyperparameter K, each policy response is first capped at K parsed invariant clauses; let $\bar{A}_i$ denote that ordered prefix and $X_0 = \bigcup_i \bar{A}_i$ its normalized, deduplicated clause union. Each optional repair round receives only target-free WP verdicts and grows the pool monotonically,

$$X_{t+1} = X_t \cup R_t.$$

Finally, $\mathsf{H}_P(X_t)$ denotes the subset that survives syntax checking and the Frama-C/WP Houdini fixpoint on the masked program P. The sequence

$$\mathcal{A} \;\longrightarrow\; X_0 \;\longrightarrow\; X_1, \ldots, X_m \;\longrightarrow\; \mathsf{H}_P(X_m)$$

is the shared rollout–Houdini core in Figure 1.

Training and inference differ only in how they consume this operator. Training evaluates individual rollouts, unions, ablations, and repaired pools to assign generation and refinement credit. Within a rollout it preserves response order to distinguish clauses that extend negative-trace coverage from later clauses that add nothing; across rollouts it uses union ablation only as positive complementary credit. Inference performs the same union, pool growth, and Houdini pruning without constructing reward traces, then restores Q for final verification. Training uses sampled positives as a cheap pre-filter, but soundness in both modes comes from the same Frama-C/WP Houdini implementation. Thus deployment is independent of the reward service while preserving the candidate and prover semantics optimized during training.

4 METHOD

Notation. For a verification instance (P, Q) with $P = (\phi_{\text{pre}}, B, S)$, a *rollout* $A = (a_1, \ldots, a_L)$ is the ordered sequence of parsed invariant clauses produced by one policy sample, and a *group* $\mathcal{A} = \{A_1, \ldots, A_G\}$ is the set of rollouts sampled for P in one RL step. The response cap is the hyperparameter K; $\bar{A} = (a_1, \ldots, a_{\min(L,K)})$ is the accepted prefix and $o(A) = \max(0, L - K)$ its overflow. The sampler emits positives $\mathcal{P}$ (sampled reachable loop-head states, each carrying its run's input valuation) and synthetic negative candidates $\mathcal{N}$ grouped into *trace units*: each unit is one perturbed history under a concrete input, witnessed by one or more states. $\mathsf{H}_P(X)$ denotes the subset of clause set X that survives the shared rollout–Houdini core on masked program P, and $\text{rej}(X) \subseteq \mathcal{N}$ the trace units rejected by $\mathsf{H}_P(X)$ — a unit is rejected iff some surviving clause is false at one of its witness states.

4.1 TARGET MASKING AND ACCEPTANCE

The implementation maintains two program views. The *original view* contains the input `requires` clauses and every verification target, including post-loop assertions, in-loop assertions, and function `ensures`. The *masked view* preserves the C program and `requires` clauses but removes complete `assert`/`ensures` clauses, including quantified targets. Generation, refinement feedback, and Houdini pruning operate only on the masked view. The sampler independently parses the loop and never consults the stored target.

The policy output is a list of scalar ACSL clauses of the form `loop invariant <expr>;`. Only the first K parsed clauses of each generation or refinement response are admitted; later clauses never enter a prover pool. These clauses are inserted before the loop together with the inferred `loop assigns` set. Candidate acceptance has two stages. First, Frama-C/WP checks one establishment and one preservation obligation per invariant; Houdini iteratively removes failed clauses. Second, the surviving set is inserted into the original view. The final result is successful only if the source parses, every expected invariant obligation succeeds, and every non-invariant WP goal induced by the restored assertions and postconditions succeeds. Thus the target is hidden from all model-facing stages but remains authoritative for evaluation.

4.2 THE SHARED ROLLOUT–HOUDINI CORE

Training and inference share one candidate-processing operator. Given a rollout group $\mathcal{A} = \{A_1, \ldots, A_G\}$, both cap each response, normalize and deduplicate its accepted clauses, form the union $U = \bigcup_i \bar{A}_i$, and use the same Frama-C adapter to compute inductive survivors. For rollout ablation, $U_{-i} = \bigcup_{j \neq i} \bar{A}_j$ is reconstructed from the other rollouts; this preserves a shared clause when another rollout also emitted it. The authoritative operator $\mathsf{H}_P(X)$ first uses a Frama-C syntax scrub to remove the exact malformed invariant lines, then runs Houdini over Frama-C/WP (Flanagan & Leino, 2001; Kirchner et al., 2015) to iteratively remove clauses whose establishment or preservation goals fail. During reward computation, a positive pre-filter first removes out-of-scope clauses and clauses falsified by sampled reachable states. Inference supplies no sampled states and skips this optimization. Because both modes end in the same prover-backed Houdini operator, sampled states improve training efficiency but do not establish soundness.

The two modes differ only in how they consume this common operator. Generation training evaluates $\mathsf{H}_P(\bar{A}_i)$, $\mathsf{H}_P(U)$, and, when the marginal term is enabled, the ablations $\mathsf{H}_P(U_{-i})$. Refinement training starts from a merged pool X and compares $\mathsf{H}_P(X \cup R_i)$ with $\mathsf{H}_P(X)$ for sampled repairs R_i. Inference forms the same union, optionally repeats the same stateless WP-verdict prompt and monotone update $X \leftarrow X \cup R_i$, and returns $\mathsf{H}_P(X)$ to the final target-bearing verifier. Thus the reward service and deployment executable remain operationally independent, but rollout grouping, pool semantics, feedback format, and Houdini survival are shared by construction.

4.3 SAMPLER: POSITIVES AND AUDITED NEGATIVE CANDIDATES

The sampler is deliberately blind to any postcondition or assertion: it executes only the precondition-respecting loop behavior needed for trace collection. The current parser accepts one braced `while` loop over scalar C `int` parameters, locals, and file-scope variables; unsupported multi-

loop/`for`/pointer inputs fail explicitly, while incomplete body state or calls trigger a positive-only fallback. It instruments the loop to print the loop-head valuation on entry, for a dense 512-iteration prefix followed by periodic eight-state bursts, and at exit; compiles with `gcc`; and executes over a deterministic, precondition-checked input sweep (edge and moderate magnitudes, systematic tier combinations, plus a few large-magnitude probes that widen the input envelope). Loops run to their real exit; runs that hit the iteration cap are flagged and excluded from any range conclusion. The observed loop-head states are the positives.

The sampler constructs negative *candidates*, not a proof of global unreachability. Family-specific evidence and conservative guards reduce label risk, while the full-suite collision audit in §5 checks the emitted labels against broader executions.

- **Relational perturbations** apply $\pm\{1, 2\}$ single-axis and pairwise steps to trace states whose next three coordinates were printed. Candidates colliding with any sampled reachable valuation are removed; the survivors expose missing laws such as $x + y = n$.
- **Over-runs** execute the real body past an observed genuine exit, preserving transition relations while moving beyond that run's exit; each continuation forms one trace unit.
- **Escapes** move one variable beyond its sampled range by ladder steps. Together with over-runs they probe missing range facts without claiming that a finite sample covers all executions.

Construction guards reduce label risk — a mislabeled (actually reachable) negative would distort the tightness signal. States observed in any run are never negatives; states that could constitute a fresh loop entry for admissible inputs (parameters free subject to `requires`, initialized locals at their initializers) are dropped; and a loop whose guard or body is controlled by `unknown()` emits positives but no synthetic negatives because finite oracle runs under-approximate its reachable set. The same no-negative fallback applies to untracked block-local state, pointer/array state, and function calls in the body. For deterministic loops, a capped run additionally disables escapes because its sampled range is incomplete. The sampler thus degrades conservatively: where state/control coverage is incomplete it emits no synthetic negatives rather than guessing labels.

4.4 Training the Shared Core

Generation training separates cross-rollout credit from within-rollout redundancy control. Standalone strength and cross-rollout complementarity are

$$\mathrm{base}(A_i) = \frac{|\mathrm{rej}(\bar{A}_i)|}{|\mathcal{N}|}, \tag{1}$$

$$\mathrm{marg}(A_i) = \frac{\left|\mathrm{rej}(U) \setminus \mathrm{rej}(U_{-i})\right|}{|\mathcal{N}|}. \tag{2}$$

Here $\mathrm{rej}(X)$ is evaluated only after applying $\mathsf{H}_P(X)$, so both reward terms use the same prover-backed survivors as inference. Base measures how much of the sampled candidate space the rollout alone excludes. Marginal is strictly positive cross-rollout credit: it pays only for negative traces lost when this rollout is removed from the group union, in the spirit of GRPO's group-relative comparison (Shao et al., 2024). It never imposes a cross-rollout redundancy penalty.

Inside each rollout, we perform an ordered coverage audit. Let $S_A = \mathsf{H}_P(\bar{A})$. For the j-th accepted clause, define

$$C_j(A) = \begin{cases} \{n \in \mathcal{N} : a_j \text{ is false at a witness of } n\}, & \mathrm{norm}(a_j) \in S_A, \\ \varnothing, & \text{otherwise}, \end{cases} \tag{3}$$

$$D_j(A) = C_j(A) \setminus \bigcup_{k<j} C_k(A). \tag{4}$$

The traversal preserves model-output order. A clause is *essential* when $D_j(A) \neq \varnothing$, and fully redundant when $D_j(A) = \varnothing$. Thus an earlier clause is marked informative as soon as it extends coverage, whereas a later duplicate, tautology, Houdini reject, or clause covered by the prefix is penalized. Let

$$e(A) = \sum_{j \le \min(L,K)} \mathbf{1}[D_j(A) \neq \varnothing], \qquad d(A) = \sum_{j \le \min(L,K)} \mathbf{1}[D_j(A) = \varnothing].$$

For example, if three clauses reject trace sets $\{1,\dots,40\}$, $\{1,\dots,40\}$, and $\{41,\dots,50\}$ in that order, their incremental coverages have sizes 40, 0, and 10; hence $e(A) = 2$ and $d(A) = 1$. The reported precision $e(A)/L$ is diagnostic and is not multiplied into the reward.

The complete generation reward is

$$r_{\text{gen}}(A) = w_b \,\text{base}(A) + w_m \,\text{marg}(A) - w_r \, d(A) - w_o \, o(A), \tag{5}$$

where $w_b, w_m, w_r, w_o \geq 0$ are reward weights. The cap makes clauses after position K ineligible for coverage credit; when $w_o > 0$, the linear overflow term makes continued invariant emission strictly costly. For the marginal ablation, the implementation sets $\text{marg}(A) = 0$ and skips every $\mathsf{H}_P(U_{-i})$ call; all within-rollout terms remain unchanged.

Rejection tests are pure Python; division and remainder use C-style truncation on the concrete integer states, while the only heavyweight cost is the prover. The distinct clause sets a group needs (per-rollout, union, ablations) are memoized and verified in parallel, and each WP run discharges its goals concurrently. The group additionally receives $\text{batch} = |\text{rej}(U)|/|\mathcal{N}|$ and a re-roll flag when it falls below a threshold. If no negative trace unit can be constructed safely, rollout and batch signals fall back to Houdini-survival fractions; the overflow cost still applies.

Refinement training is a separate group type, not a third generation-reward term. Given a shared merged pool X and repair response R_i, its reward is

$$\Delta\text{base}_i = \text{base}(X \cup R_i) - \text{base}(X), \tag{6}$$

$$r_{\text{ref},i} = \Delta\text{base}_i - w_o \, o(R_i). \tag{7}$$

Here R_i denotes the capped response prefix when joined to X; the existing pool itself is not truncated. The pool only grows, so the diagnostic delta is nonnegative. Within the cap, copied, trivial, or broken repairs that add no discrimination after H_P receive zero; an overlong response can receive negative training reward through its overflow cost. Generation and refinement groups are mixed in one policy update, directly training the two model actions exercised by deployment.

4.5 Inference through the Shared Core

At inference time the rollout provider receives the masked program and returns n candidate invariant sequences, each capped at K clauses before union. The framework applies the same normalization and union operation used in training; unlike reward computation, inference never invokes the execution sampler or constructs positive and negative examples. When $m > 0$, each refinement round runs a syntax check and at most two Frama-C/WP passes on the masked program. The second pass is needed only after removing establishment failures that could mask preservation failures. The resulting verdict table reports syntax and scope errors and distinguishes failed establishment from failed preservation, but contains no target-derived feedback. The stateless prompt is the same template used to form refinement-training groups and asks the provider to repair rejected clauses or add supporting companions; each refinement response is likewise capped at K clauses before joining the pool. New candidates join the pool and originals remain, so a later companion may rescue an earlier clause. Refinement stops when no clause is rejected, the pool reaches a fixpoint, or the m-round budget is exhausted; $m = 0$ recovers the plain pipeline.

The shared H_P operator then performs full Houdini pruning and produces the final invariant set on the masked view. Only then does the framework annotate the original target-bearing source and run Frama-C/WP. The verifier separately checks that the number of invariant results matches the number of inserted clauses and that all assertion, postcondition, and contract goals succeed. If final verification fails, inference may request another rollout group up to its re-roll budget; candidate ranking prefers a verified result and then the result with more surviving invariants. Because refinement only enlarges the pool and the same final proof gate remains authoritative, enabling refinement cannot remove a clause accepted by the plain pipeline.

5 Experiments

5.1 Benchmarks and Evaluation Contract

Core-366. The primary suite contains 366 single-loop C programs: 316 linear programs drawn from Code2Inv, SyGuS, and SV-COMP sources, and 50 nonlinear programs from the LIPuS benchmark.

The latter includes polynomial identities, products, extended GCD, and division-by-subtraction algorithms. Assertions are present in 356 programs and absent in 10. They remain in the final verification instance but are removed from every model prompt and intermediate prover query. The repository also contains the 466 integer programs from Loopy's original 469-program corpus; its three floating-point tasks are outside our predicate language. Because the two corpora share upstream sources, we treat Loopy-466 as a comparison suite rather than 466 independent training examples.

Controlled target-hidden comparison. Published baseline scores are not eligible for the main comparison because those systems solve the complete verification problem and may use Q in generation, ranking, constraint construction, or repair. We instead rerun target-hidden adaptations of Loopy, Clause2Inv, and IRANK. Every method receives the same masked program $P \setminus Q$; neither its policy nor any intermediate oracle may observe Q or a verdict derived from Q. Loopy-no-Q receives only invariant initiation and preservation feedback. Clause2Inv-no-Q constructs and combines clauses using only initiation and consecution constraints, excluding safety counterexamples and Q-derived constraints. IRANK-no-Q ranks a fixed target-hidden candidate pool without target text or target-derived verifier features. Only after each method has fixed its output do we restore Q and run the same final Frama-C/WP check.

Primary endpoint and budgets. For a method M, let $I_M(P \setminus Q)$ be the invariant set produced without target access. The headline target-hidden accuracy is

$$\mathrm{Acc}_{\mathrm{hidden}}(M) = \frac{1}{|\mathcal{D}|} \sum_{(P,Q)\in\mathcal{D}} \mathbf{1}\big[\mathrm{WP}\big(P, I_M(P \setminus Q), Q\big) = \mathrm{valid}\big] .$$

Parse failures, timeouts, and unsupported attempted inputs count as failures; the denominator is therefore identical across methods on each reported suite. We separately report the *exit gap*: programs with inductive Houdini survivors whose restored target still fails. This distinguishes producing valid loop facts from producing an exit-sufficient summary. Efficiency metrics are model calls, generated tokens, full-Houdini calls, final-WP calls, and wall time. All neural comparisons must use the same checkpoint, prompt, temperature, output-token limit, and response budget; every generation and refinement response is capped at the same $K = 20$ parsed invariant clauses. We use $G = 8$ initial rollouts and compare $m \in \{0, 1, 2, 4\}$ refinement rounds; an equal-call re-generation control receives exactly m additional fresh generations.

5.2 End-to-End Comparisons and Ablations

The controlled evaluation is organized around three questions. First, under target hiding we compare Direct-no-Q, Loopy-no-Q, Clause2Inv-no-Q, and IRANK-no-Q against union plus Houdini, equal-budget re-generation, shuffled WP feedback, and the full m-round LoopGym refinement loop (Chakraborty et al., 2023). Generated candidates are held fixed for the ranking comparison, while model calls are held fixed for refinement versus re-generation. Failure is assigned to one of six exclusive buckets: no parseable clause, syntax/scope error, initiation failure, preservation failure, inductive-but-insufficient, or tool timeout.

Second, the same base model is run with and without the target only as a *shortcut diagnostic*, not as a baseline comparison. A literal copy-Q test measures how often the assertion itself is inductive, and exact normalized target copies in model outputs are counted in both conditions. This diagnostic quantifies leakage from target visibility, while every headline accuracy remains target-hidden.

Third, training compares the base checkpoint, the full generation reward in Eq. (5), its explicit no-marginal variant (`use_marginal=false`), and the full mixed generation–refinement objective. Further shaping ablations set the ordered redundancy cost w_{red} or overflow cost w_{over} to zero while leaving the 20-clause admission cap fixed. We also remove each negative family in turn. The key paired comparison is the same trained policy with $m = 0$ and $m > 0$: it tests whether the stateless repair action learned inside the shared framework transfers to repeated inference. We use whole provenance/clone clusters for train/validation/test assignment, rather than random files, to prevent the Code2Inv/SyGuS/SV-COMP overlap from inflating transfer. The complete run matrix, raw JSONL schema, and table templates are included in the artifact's `paper/EXPERIMENT_PLAN.md`.

5.3 Measured Reward Regression Checks

The current artifact contains no trained checkpoint, so we do not fill the end-to-end table with synthetic numbers. It does contain two measured checks of the signal on which training depends.

Discrimination harness. Ten hand-written specifications span long linear loops, symbolic bounds, division, polynomial relations, products, and extended GCD. Each program is scored with known-quality families: gold, loose, trivial, guard-copy, post-copy when an assertion exists, and an unsound sampled-value pin. The harness requires every gold clause to survive, gold base rejection to be at least 0.8, and the shortcut/unsound families to remain at least 0.15 below gold unless a postcondition is genuinely inductive. The measured real-Frama-C run completed all 10 specifications with **0 violations**; every gold clause survived and the minimum gold base score was **0.996**.

Full negative-label audit. For every Core-366 program, the audit constructs negatives with the canonical 12-run, seed-0 sample and looks for the same variable/pre-state pair in larger 24-run samples under seeds 0 and 9. The measured run covered **486,302** negative witness states and found **0 hard pair-level collisions**. It reported 269 variable-only overlaps across 18 programs where the associated pre-state differed; these are soft diagnostics rather than label collisions.

Scope. These checks validate reward discrimination and sampled-label consistency, not model quality. A complete empirical claim still requires the end-to-end runs above, their raw rollouts and prover logs, and multiple training seeds. Until those artifacts exist, we report no target-hidden baseline accuracy; the measured reward checks remain deliberately separated from model-quality claims.

6 Conclusion

Loop verification requires invariants that are both inductive and strong enough to establish the condition required after the loop. When that condition is visible, generation may collapse to copying the assertion, as illustrated by `NLA_lipus/10.c`. We instead study target-hidden verification: the policy sees the loop but not its assertion, while the original assertion remains the final acceptance criterion.

LoopGym centers training and inference on the same rollout–Houdini framework. Both paths sample groups of candidate invariant sets, merge and optionally repair a growing clause pool, and retain its inductive core with the same Frama-C/WP Houdini operator. The training objective assigns dominant standalone rejection, positive cross-rollout marginal credit, and an ordered within-rollout redundancy cost to generation. Only clauses with zero incremental negative coverage are penalized, and a configurable K-clause response cap with overflow cost prevents unbounded output growth. Refinement receives pool-improvement credit subject to the same response cap; inference replaces these rewards with final target-bearing verification. This shared scaffold aligns learning with deployment while keeping Q absent from prompts, feedback, pruning, and reward labels. The pipeline therefore rewards semantic information carried through the loop without weakening the end goal: a result succeeds only when the restored postcondition verifies.

Beyond the immediate system, we see the shared training–inference scaffold and audited behavioral rewards as portable. Audited synthetic negatives generalize to domains where construction-specific evidence is cheaper than proving optimality; keeping assumptions and collision checks explicit is preferable to compensating for bad labels in the scoring formula. Future work includes multi-loop and heap-manipulating programs (Liu et al., 2024), richer assertion languages, and conditions that connect several loops before a final postcondition.

References

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.

Carlo Alberto Furia and Bertrand Meyer. Inferring loop invariants using postconditions. In *Fields of Logic and Computation: Essays Dedicated to Yuri Gurevich*, pp. 277–300. Springer, 2010.

Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pp. 499–512, 2016.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pp. 683–693, 2012.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026.

Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.

A Canonical Component Defaults

Table 1 lists the implemented defaults. Execution-trace sampling belongs only to reward construction, while grouped model rollouts, monotone refinement pools, and the Houdini backend are shared with inference. Inference has its own refinement budget and never constructs a trace example set.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 values in $[-64, 64]$	deterministic edge-first magnitude schedule
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	$\pm\{1, 2\}$	single-axis and pairwise perturbations off dense bases
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
escape ladder	$\pm\{5, 8, 13, 21, 34\}$	candidates kept only outside the sampled range
over-run steps	24	post-exit continuation length (one trace unit per run)
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_b, w_m)	$(0.8, 0.2)$	standalone / cross-rollout marginal weights
w_r	0.02	cost per clause with zero ordered incremental coverage
K	20 clauses	response cap applied independently to every generation/refinement response
w_o	0.05	overflow cost per clause after K
marginal switch	enabled	disabling it zeros the term and skips rollout ablations
re-roll threshold	0.6	group batch score below which re-rolling is advised
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget
m_{refine}	0	inference refinement disabled by default; any non-negative round budget is supported

Table 1: Implemented defaults of the sampler (§4.3), reward (§4.4), inference, and verification backend.

B Generation Prompt

The complete generation template; closed-book inference strips an assertion when one is present before inserting the program source. General ACSL rules are supplied separately by `system_prompt.txt`.

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

C REWARD SERVICE INTERFACE

The reward framework is packaged as a self-contained, CPU-only container with gcc, Frama-C/WP, Why3, and Z3 bundled. It exposes `POST /reward`, accepting program source and a rollout array, with the following response fields:

```
{program, n_positives, n_negatives, filter_mode,
 rollout_rewards[], base[], marginal[], marginal_rejected[],
 redundant_clauses[], redundancy_penalty[], essential[], precision[],
 generated[], accepted[], overflow[], overflow_penalty[],
 marginal_enabled, rollouts[], batch_score, should_reroll}
```

Rollouts may be raw LLM text, invariant lists, or annotated code; parsing is uniform. The request flag `use_marginal=false` implements the marginal ablation without paying for leave-one-rollout-out prover calls. For refinement training, `POST /refine_feedback` returns the WP verdict table and assembled stateless prompt, while `POST /refine_reward` returns both the nonnegative diagnostic Δbase and `refine_rewards[]` after overflow cost. An offline scorer provides the same generation reward over JSONL/Parquet rollout dumps and exposes `--disable-marginal`.